Implementing the State Pattern in Actor Framework

The **state pattern** is a behavioral design pattern that "allow[s] an object to alter its behavior when its internal state changes. The object will appear to change its class."[1] Use this pattern if an object must change its behavior based on its internal state (i.e. if the class has two or more modes of operation such that the mode change alters the behavior of most of the class methods). This pattern lets you avoid adding case structures that test for mode to all of your VIs, generally making for more maintainable code. This package demonstrates a straightforward way to create a family of actors that together implement the state pattern. We will refer to this collection of actors as a State Pattern Actor.

Actor Framework 4.1 introduces a new dynamic dispatch VI, **Substitute Actor.vi**. This VI allows us to replace an already launched actor with another actor. You don't have to launch the new actor, and you access it using the same enqueuer. Substitute Actor works by transferring the caller and self queues from the original actor to the new actor (the original actor is destroyed). You can use an override of Substitute Actor to pass on other shared data. There is an important limitation: the core of the originally launched actor continues to run, with the new actor as its internal data. In practical terms, we must observe the following:

1. All of the state actors must share a common ancestor more specific than just Actor.lvclass.
2. Only that common ancestor class may override **Actor Core.vi.**

The State Actor

This package adds a new actor, **State Actor.lvclass**, to your user.lib/Actors folder. State Actor is an abstract class that provides a framework for defining the behavior of state transitions. State Actor includes the following methods.

Entry.vi: This is an abstract method. Override this method to define the behavior when you enter the state.

Exit.vi: This is an abstract method. Override this method to define the behavior when you leave the state.

Change State.vi: This method defines state transitions; use it to construct the specific state transitions of your State Pattern Actor. The method takes the current state and the next state as inputs. It invokes **Exit.vi** on the current state. It then invokes **Substitute Actor.vi** to transfer data from the current state to the next state. Finally, it invokes **Entry.vi** on the new state, and returns that state as the current state.

Actor Core.vi: This override invokes **Entry.vi** on the initial state, prior to calling the parent Actor Core. Adding this behavior to Actor Core (instead of an override of **Pre Launch Init.vi**) allows the initial state to launch a nested actor as part of its entry behavior.

---

[1] Gamma, Helm, Johnson, and Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, 1994, p. 305

Building State Patter Actors

In an Actor Framework implementation of the state pattern, you create a context actor which inherits from State Actor. The context actor will be the common ancestor of all the actors that make up your State Pattern Actor, and it is the only actor that may override Actor Core.vi. You then create children of the context actor, one for each state in your system. The context actor may itself represent a valid state of the system, or may simply be an abstract class.

You will create the following for your context actor:

1. Any attributes to be shared by all the states of the system.
2. An override of **Substitute Actor.vi** that will transfer those attributes from the current state actor to the new state actor.
3. The override of **Actor Core.vi** (if any).
4. A dynamic dispatch method for every method whose behavior changes based on state.
5. A method for every state change the actor's caller can invoke. (State transitions can also be invoked by other methods of the context actor. For example, any method might include the code to transition to the "System Faulted State" if an error occurs).
6. Message classes to invoke those methods.

Each method that changes state will invoke **Change State.vi**, a protected method of State Actor. Wire a class constant for the new state actor to the **New State** input. Wire **State in** to your method's context actor control, and **State out** to the context actor indicator**.**

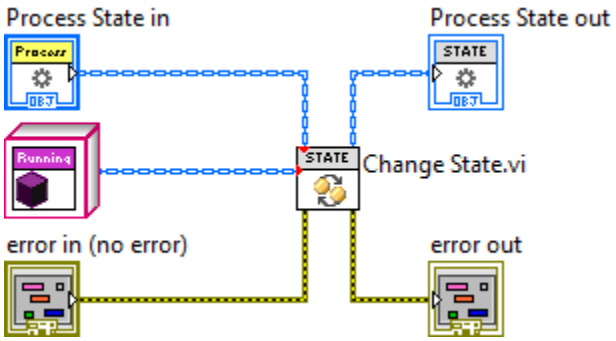

Figure 1: A State Transition Method

Each state in your system is a child of the context actor. For each of these state actors, you will provide the following:

1. Overrides of **Entry.vi** and **Exit.vi** that provide the transition behavior for this state.
2. Overrides of methods whose behavior will change when in the state.
3. Any additional methods for behaviors unique to the state.
4. Messages to invoke the unique public behaviors of the state, if the state adds any public methods beyond those inherited from the parent.

You may also choose to include override methods of the context actor's state transition methods. You can do this to add behavior to specific state transitions, i.e. do X when we transition from State A to State B. You can also do it to disable the default transition behavior, which may be desirable when exiting and reentering a state (i.e. when we transition from State A to State A).

If you need to add behavior to a specific state transition, and that behavior must occur after exiting State A but before entering State B, you may add the behavior to an override of **Substitute Actor.vi**. Note that the override is specific to the state you are exiting; you will need to create a mechanism to invoke code specific to the new state.

If you create a message to invoke the unique method of a specific state, you will generate an error if you send the message to the State Pattern Actor when it is in a different state. If you do not wish to generate this error, then you must guarantee that all states in the system will gracefully handle the message. The easiest way to gracefully handle such a message is to silently discard it if the message doesn't apply to the current state. You can do this by modifying the message's Do method as shown in Figure 2. If you want more complex behavior, strongly consider moving the method up to the parent class so that each state may override as needed.

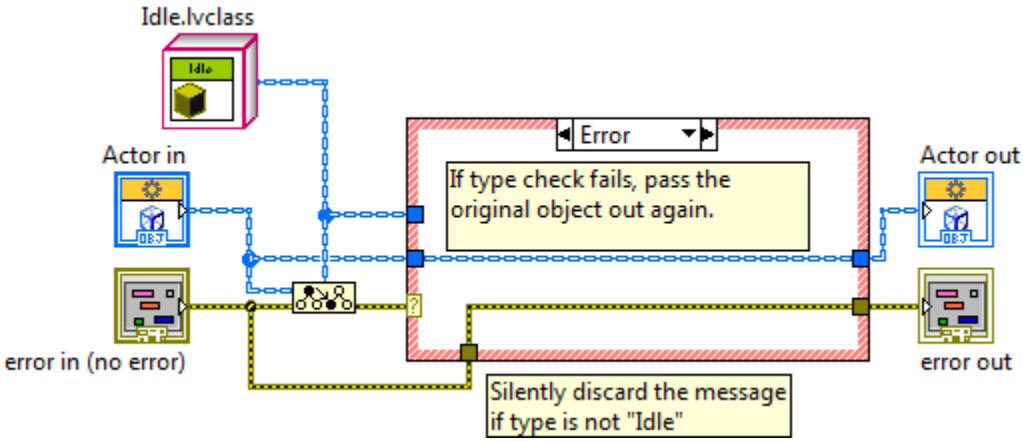

Figure 2: Discarding a Message

You launch the system by passing an initial state to **Launch Actor.vi**, as shown.

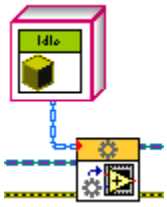

Figure 3: Launching a State Pattern Actor

The system's caller may trigger a state transition by sending one of the public state transition messages. A state may trigger a transition from one of its own methods by invoking a transition method directly.

Running the Example

The package adds the State Pattern Actor project to the list of sample projects. Select the project and create a local copy.

Open and run **Load App.vi**. The State Pattern UI will open. Note that the event log shows we have entered the Idle state.

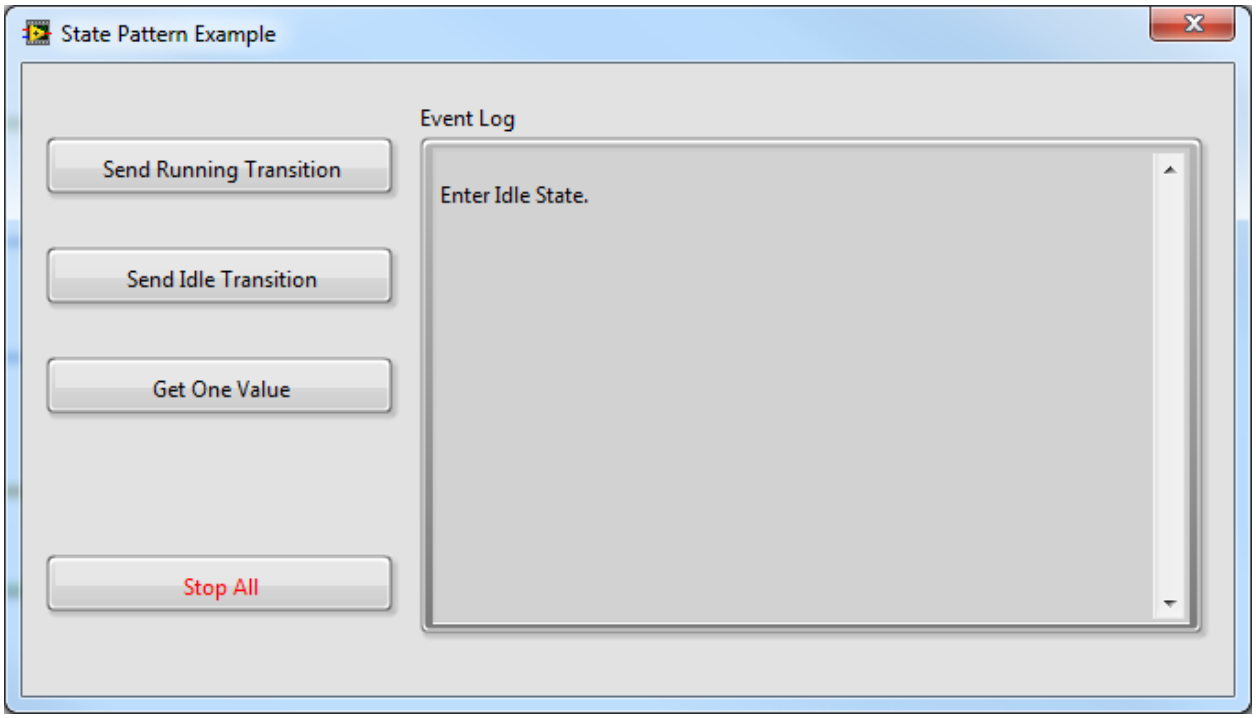

Figure 4: The State Pattern UI

Click on **Get One Value** and observe the results. Click on **Send Running Transition.** The system acquires several data points and performs a calculation. Note that the system exits the Running state and enters the Idle state automatically. At this point, your front panel should resemble this:

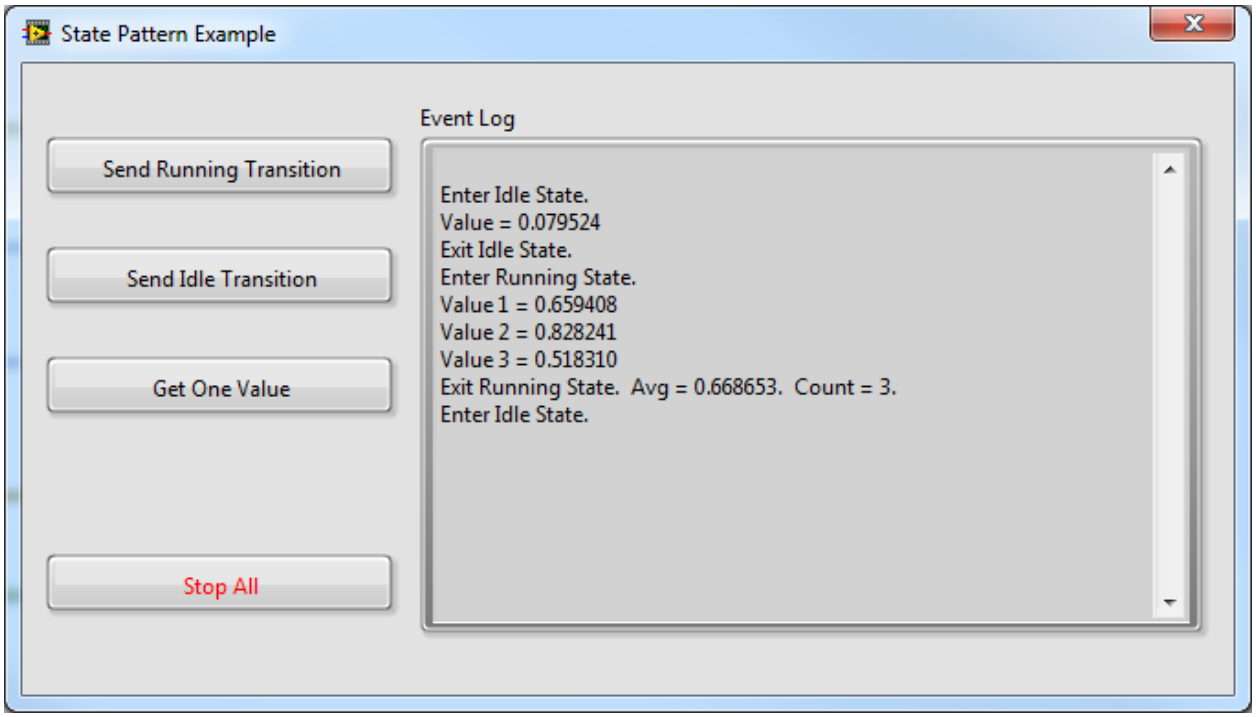

Figure 5

Click on **Send Idle Transition**, and note that nothing happens. The system is already in Idle, and no transition is necessary. Now click on **Send Running Transition**, wait until the first value appears, and click on **Send Idle Transition.** Note that the system quits the acquisition early, returning a partial data set, and transitions to Idle.

The Example, Explained

The example was built using the Actor Framework Template. It consists of a launcher, a user interface, an actor that simulates a data acquisition process, and a family of actors that comprise a State Pattern Actor.

Open and run **Load App.vi** to launch the example.

State Pattern UI.lvlib contains the user interface actor for the example, **State Pattern UI.lvclass**. This actor launches a State Pattern Actor, Process States, as a nested actor.

Process Actor.lvlib contains an actor that simulates a data acquisition process. You can send this actor an Update message, which will cause the actor to generate a random number and send it to its caller. If you send this actor a Start Acquisition message, it will generate and send a random number every second, until you send it a Stop Acquisition message.

Process States.lvlib contains the actors and messages that comprise a State Pattern Actor. The actor launches a single Process Actor as a nested actor. **Process State.lvclass** is the context actor. **Idle.lvclass** and **Running.lvclass** are the two supported states.

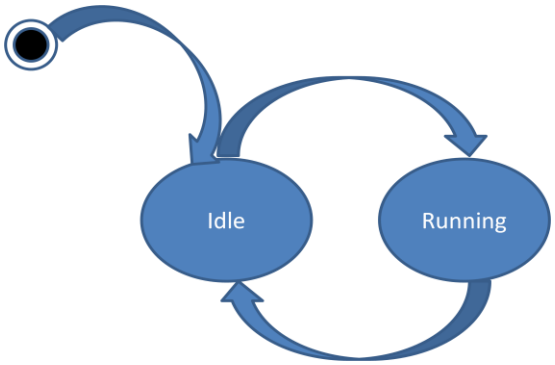

Figure 6: Supported States

Process States accepts these messages:

1. **Go to Idle**: If the current state is Running, the actor will immediately transition to Idle. The message has no effect if the current state is Idle.
2. **Go to Running**: If the current state is Idle, the actor will immediately transition to Running. The message has no effect if the current state is Running.
3. **Request Update from Process**: If the current state is Idle, the actor will request a single data point from its Process Actor. The message has no effect if the current state is Running.
4. **Update from Process**: Process Actor sends data to Process State using this message.

Process State.lvclass is the context actor. It inherits from State Actor and includes:

1. An override of **Actor Core.vi** that launches its Process Actor.
2. An override of **Substitute Actor.vi** that transfers Process Actor's enqueuer from the current state to the new state.
3. Two state transition methods, **Go to Idle.vi** and **Go to Running.vi.**
4. An abstract method, **Update from Process.vi**, that receives data from Process Actor.

Idle.lvclass is the actor for the Idle state. It inherits from Process State and includes:

1. An override of **Entry.vi** that sends the "Entering Idle State" message to the UI.
2. An override of **Exit.vi** that sends the "Exiting Idle State" message to the UI.
3. An override of **Go to Idle.vi** that *disables* the call to **Entry.vi**.
4. An override of **Update from Process.vi** that sends a single data point to the UI for display.
5. A unique method, **Request Update from Process.vi**, which sends an Update message to the Process Actor.

When you click on the **Get One Value** button on the UI, it sends a Request Update from Process message to Process State. Note, however, that only Idle can respond to that message. This message was modified as described above to be dropped if received by Process State or Running.

Running.lvclass is the actor for the Running state. It also inherits from Process State and includes:

1. An override of **Entry.vi** that sends a Start Acquisition message to Process Actor and the "Entering Running State" message to the UI.
2. An override of **Exit.vi** that sends a Stop Acquisition message to the Process Actor and the "Exiting Running State" message and the average value and cont data to the UI.
3. An override of **Go to Running.vi** that *disables* the call to **Entry.vi**.
4. An override of **Update from Process.vi** that sends a single data point to the UI for display. This override also increments the value count by one and calculates a running average value.

Idle and Running interact with Process Actor and State Pattern UI differently. They send different messages and behave differently when they receive messages. Idle responds to more messages than Running.

This is the complete system.

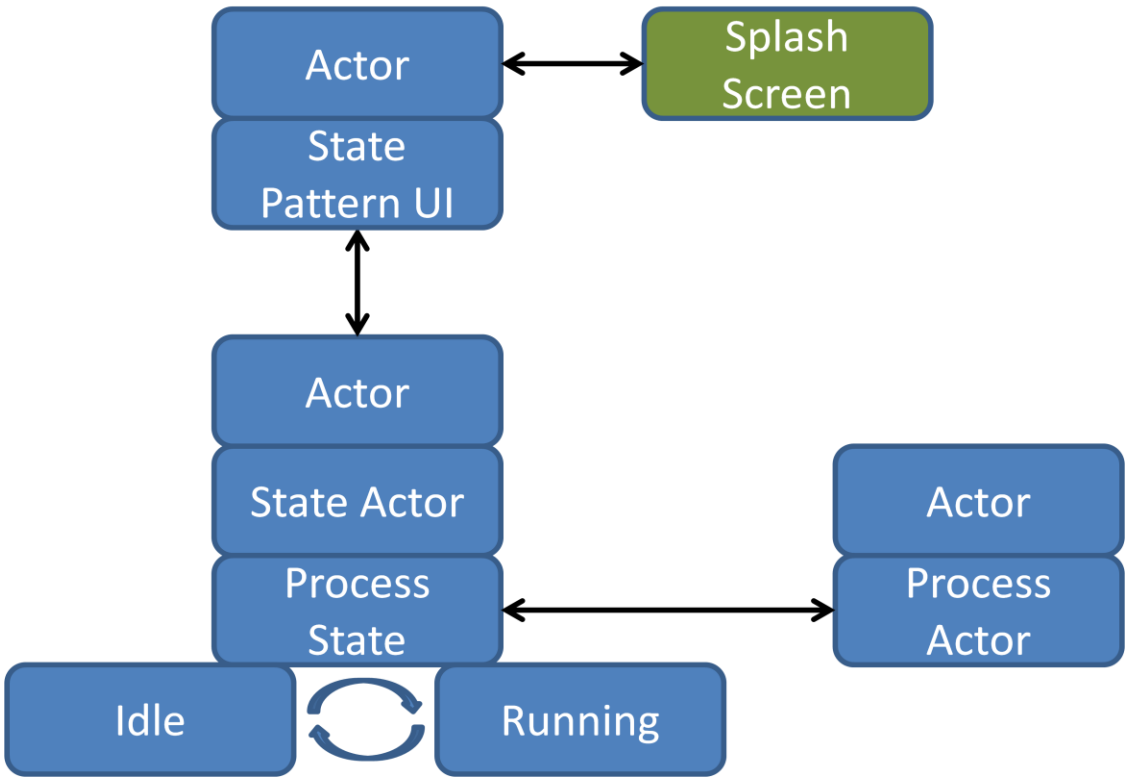

Figure 7: The State Pattern Example